

C# - PHẦN 9.4

Func<T> - Generic Delegate có giá trị trả về

Ý chính: Action dùng cho method không trả về giá trị; Func dùng cho method có trả về giá trị.

1. Từ Action sang Func

Ở bài trước, Action<string> chỉ dùng được với method trả về void. Ví dụ:

```
Action<string> action = NotificationService.SendEmail;
```

Nếu method trả về một giá trị, chẳng hạn:

```
public static int GetLength(string text)
{
    return text.Length;
}
```

thì không thể dùng Action<string>. Lúc này ta dùng:

```
Func<string, int>
```

2. Cách đọc Func

Ví dụ:

```
Func<string, int>
```

Hãy đọc: nhận một string và trả về int. Tương ứng với chữ ký:

```
int Method(string value)
```

Ví dụ thực tế:

```
public static int GetLength(string text)
{
    return text.Length;
}

Func<string, int> getLength = GetLength;

int result = getLength("Hello");
Console.WriteLine(result); // 5
```

Quy tắc quan trọng nhất: kiểu cuối cùng trong Func<...> luôn là kiểu trả về.

3. Func<int>

Func<int> không có parameter và trả về int. Có thể hình dung như:

```
int Method()
```

Ví dụ:

```
public static int GetNumber()
{
    return 100;
}

Func<int> getNumber = GetNumber;
Console.WriteLine(getNumber());
```

4. Func<int, int>

Func<int, int> có một parameter int và trả về int. Tương ứng:

```
int Method(int value)
```

Ví dụ:

```
public static int Square(int number)
{
    return number * number;
}

Func<int, int> calculate = Square;
int result = calculate(5); // 25
```

5. Func<int, int, int>

Trong Func<int, int, int>, hai kiểu đầu là parameter và kiểu cuối là return type.

```
int Method(int first, int second)
```

Ví dụ:

```
public static int Add(int first, int second)
{
    return first + second;
}

Func<int, int, int> operation = Add;
```

```
int result = operation(10, 5); // 15
```

6. So sánh Action và Func

Delegate	Parameters	Return
Action	0	void
Action<string>	string	void
Action<int, string>	int, string	void
Func<int>	0	int
Func<string, int>	string	int
Func<int, int, bool>	int, int	bool

Cách nhớ: Action làm một hành động và không trả kết quả; Func xử lý/tính toán và trả về kết quả.

7. Liên hệ với CalculateDelegate cũ

Ở bài 9.1, ta từng tự khai báo:

```
public delegate decimal CalculateDelegate(  
    decimal first,  
    decimal second);
```

Delegate này nhận hai decimal và trả về decimal. Với Func, có thể viết:

```
Func<decimal, decimal, decimal>
```

Ví dụ:

```
Func<decimal, decimal, decimal> operation =  
    Calculator.Add;  
  
decimal result = operation(10, 5);
```

Về mặt chữ ký, CalculateDelegate và Func<decimal, decimal, decimal> có thể đại diện cho cùng loại method.

8. Truyền Func vào một method khác

```
public static decimal Calculate(  
    decimal first,  
    decimal second,  
    Func<decimal, decimal, decimal> operation)  
{  
    return operation(first, second);  
}
```

Gọi:

```
decimal result = Calculate(  
    10, 5, operation);
```

```
10,  
5,  
Calculator.Add);
```

Bên trong, `operation(first, second)` thực chất sẽ gọi `Calculator.Add(10, 5)`. Đây vẫn là tư duy Delegate: truyền một hành vi/method vào method khác.

9. Func trả về bool

```
public static bool IsAdult(int age)  
{  
    return age >= 18;  
}  
  
Func<int, bool> checkAge = IsAdult;  
  
bool result = checkAge(20);  
Console.WriteLine(result); // True
```

`Func<int, bool>` có nghĩa là nhận `int` và trả `bool`. Kiểu này sẽ xuất hiện rất nhiều khi học Lambda và LINQ.

10. Ví dụ Func với object

```
public class Product  
{  
    public string Name { get; }  
    public decimal Price { get; }  
  
    public Product(string name, decimal price)  
    {  
        Name = name;  
        Price = price;  
    }  
}
```

```
public static decimal GetPrice(Product product)  
{  
    return product.Price;  
}  
  
Func<Product, decimal> priceSelector = GetPrice;  
  
Product product = new Product("Laptop", 1500);  
decimal price = priceSelector(product);
```

`Func<Product, decimal>` nghĩa là nhận `Product` và trả `decimal`.

11. Func cũng là Delegate

Vì `Func` là Delegate nên về kỹ thuật nó vẫn có thể dùng multicast:

```
Func<int, int> operation = Square;
```

```
operation += AnotherMethod;
```

Tuy nhiên, với multicast delegate có return value, khi gọi trực tiếp thì giá trị nhận được là giá trị trả về của method cuối cùng trong invocation list. Vì vậy Func thường được dùng để đại diện cho một hành vi trả về kết quả, chứ không thường dùng multicast như Action.

12. Func không phải Generic Method

Ví dụ:

```
Func<string, int>
```

đây là một Generic Delegate có sẵn của .NET, không phải Generic Method. Còn ví dụ sau mới là Generic Method:

```
public static T GetFirst<T>(List<T> items)
```

13. Cách đọc nhanh Func

- Func<int> -> không nhận gì, trả int.
- Func<string, int> -> nhận string, trả int.
- Func<int, bool> -> nhận int, trả bool.
- Func<int, int, decimal> -> nhận int, int và trả decimal.
- Func<Product, string> -> nhận Product và trả string.

Phản xạ cần nhớ: nhìn Func từ trái sang phải, nhưng kiểu cuối cùng luôn là return type.

14. Tại sao Func quan trọng?

Sau này bạn sẽ gặp các kiểu như Func<Product, bool> hoặc Func<Student, string> rất nhiều. Khi học Lambda, ta có thể viết:

```
Func<int, bool> check =  
    number => number > 0;
```

Và trong LINQ bạn sẽ gặp các biểu thức như:

```
products.Where(  
    product => product.Price > 1000);
```

Hiện tại chưa cần hiểu LINQ. Chỉ cần biết rằng kiến thức Func hôm nay là nền tảng để phần Lambda dễ hơn.

15. Tóm tắt bài 9.4

- Action: Delegate trả void.
- Func: Delegate có giá trị trả về.
- Func<string, int>: nhận string, trả int.
- Trong Func<T1, T2, TResult>, kiểu cuối cùng luôn là return type.

16. Bài tập 9.4 - Func

Tạo project/file theo cấu trúc:

```
FuncBasic/  
├── Calculator.cs  
├── FuncProcessor.cs  
└── Program.cs
```

Trong Calculator.cs, viết các method Add, Subtract, Multiply; mỗi method nhận hai decimal và trả decimal.

Trong FuncProcessor.cs, tạo:

```
public static decimal Process(  
    decimal first,  
    decimal second,  
    Func<decimal, decimal, decimal> operation)  
{  
    // tự hoàn thiện  
}
```

Trong Program.cs, thực hiện lần lượt:

- Tạo Func<decimal, decimal, decimal> operation trỏ tới Calculator.Add.
- Gọi operation(10, 5).
- Đổi operation sang Calculator.Subtract rồi gọi lại.
- Gọi FuncProcessor.Process(10, 5, Calculator.Multiply).
- Tạo Func<int, bool> checkAge trỏ tới method IsAdult(int age).
- Test checkAge(20) và checkAge(15).

Cuối bài, trả lời 4 câu:

- Action và Func khác nhau ở điểm cốt lõi nào?
- Func<string, int> có bao nhiêu parameter và trả về kiểu gì?
- Trong Func<int, int, decimal>, decimal có ý nghĩa gì?
- Vì sao Calculator.Add có thể được truyền vào Func<decimal, decimal, decimal>?

Mấu chốt của bài: nếu bạn hiểu "kiểu cuối cùng của Func là return type", phần còn lại sẽ khá nhẹ.